



TRUST404 · TRACK 04

AutoPoC

Autonomous Exploit-Proof Agent

취약점 "**의심**"을 보고서로 남기지 않는다.

공격이 실제로 성립함을 **실행되는 PoC로 스스로 증명**하는 에이전트.

실행 증명

캐시 결정론

오탐 0

스스로 도출

미공개 일반화

01 문제 정의

- ◆ 감사의 병목은 "확인"이다. 정적 분석·LLM 도구는 취약 가능성을 쏟아내지만, 진짜 성립 여부 판단은 사람 몫 — 느리고 비싸다.
- ◆ "가능성 리포트"는 신뢰할 수 없다. 오탐(false positive)이 섞여 있어, 리포트 한 줄마다 사람이 다시 검증해야 한다.
- ◆ 재현이 어렵다. 같은 입력에도 결과가 흔들리면 회귀 검증·감사 추적이 불가능하다.
- ◆ → 필요한 것: 공격 성립을 스스로 · 재현 가능하게 · 오탐 없이 증명하는 자율 에이전트.

02 위협 모델

입력 & 공격자 능력

타깃 컨트랙트 소스 + 불변식(*Invariants*) + manifest를 입력받는다. 공격자는 임의의 트랜잭션 시퀀스·재진입·외부 콜·자금 이동을 수행할 수 있다.

신뢰 경계

LLM은 소스 추론만 한다 — 파일·웹 접근 없음, 타깃별 정답 하드코딩 없음. 모든 후보는 하네스 실행으로만 채택된다.

성공의 정의

조직위 하네스 `!Invariants.checkAll`이 실제로 깨져야만 성공. 에이전트도 LLM도 판정 권한이 없다 — 판정은 forge 실행뿐.

안전장치

멀쩡한 타킷엔 거짓 성공을 만들지 않고 기권(exit 1). 네트워크는 허용된 LLM API 예외만 — 오프라인에선 완전 차단으로도 동작.

입력 · 타깃 소스 · **Invariants.sol** · manifest.json



TRIAGE — 정적 분류로 취약 유형(vuln_class) 판별 수 ms



ROUTER — 유형별 두뇌 우선순위 결정 오프라인 결정론 두뇌를 클라우드보다 먼저



Fuzzer

Reentrancy

Cloud LLM

Local LLM

Heuristic

지식 카드(RAG-lite)가 LLM 프롬프트에 힌트 주입 · 첫 실행-증명 후보가 채택된다

▼ 각 후보

VERIFICATION GATE — forge 하네스로 실제 실행 → **checkAll이 깨지는가?**

깨짐 → **PROVEN**

솔루션 캐시에 고정 → **Exploit.sol + attempts.log** ·
exit 0

안 깨짐 → 자가수정 ↻

실패를 피드백으로 다음 두뇌/재시도에 주입

예산 소진 → 기권

거짓 성공 없이 **exit 1**

Triage가 유형을 분류하면 그 유형에 **가장 강한 두뇌를 먼저** 돌린다. 모든 두뇌는 동일한 실행 검증 게이트 뒤에 있어, 약한 두뇌의 오답은 채택되지 않는다.

두뇌	담당(강점) 유형	방식	오프라인	결정론
Fuzzer	접근제어 · 산술	invariant 퍼징 → 축약 반례 → PoC 합성 (사전지식 0)	✓	✓
Reentrancy	재진입	외부콜-선행 함수 탐지 → 재진입 액터 합성	✓	✓
Cloud LLM	오라클 · delegatecall · 신종	소스 추론 + 실패 자가수정 · 폴백 체인 sonnet-5→opus-4-8	키 필요	캐시로
Heuristic	tx.origin · delegatecall 등	정적 공격 템플릿(결정론 최후수단)	✓	✓
Local LLM (선택)	오프라인 추론	Ollama qwen2.5-coder — 이미지에 구울 때만	✓	캐시로

"그냥 AI"도 "그냥 퍼저"도 아니다 — **각 유형의 최강 도구를 하나의 검증 게이트 뒤에 결합**. 재진입처럼 퍼저가 못 잡는 클래스는 전용 브레인, 퍼즐형은 LLM이 담당한다.

강점 01 증명되지 않으면 성공이 아니다

실행 · 오탐

■ 실행 증명 게이트

모든 후보는 **forge** 하네스로 실제 실행되어 **checkAll**을 실제로 깬 때만 채택된다.
"될 것 같다"는 후보는 산출물이 되지 못한다. 판정 권한은 하네스뿐.

■ 오탐 0 — 멀쩡하면 기권

취약하지 않은 타깃은 예산을 다 써도 **exit 1**로 기권한다. 거짓 성공(false positive)을 구조적으로 만들지 않는다.

제출되는 모든 성공은 이미 실행으로 증명된 것 — 실행·재현 실패 리스크를 산출 단계에서 제거하고, 취약/정상 컨트랙트를 **exit 0** / **exit 1**로 명확히 가른다.

■ 솔루션 캐시

성공한 PoC를 (타겟 소스 해시, 시드)로 고정. 재실행은 LLM을 다시 부르지 않고 **바이트 동일**하게 리플레이한다.

■ LLM 비의존 결정론

퍼저 시드 고정 + 오프라인 결정론 두뇌 우선. 결정론이 **LLM 온도·샘플링**에 흔들리지 않는다.

■ 검증됨

오프라인 **N=3 바이트 동일** 신규 생성 + 클라우드 캐시 리플레이 **동일**로 실측 확인.

동일 조건 → **동일한 PoC**. 생성 경로(캐시 없는 재생성)와 재현 경로(캐시 리플레이)가 모두 결정론적이라, 회귀 검증과 감사 추적이 가능하다.

스스로 도출

정답 하드코딩 없음. 클라우드 브레인은 **파일·웹 접근이 없어** 넘겨준 소스만 본다. 퍼저는 **사전지식 0**으로 반례를 탐색 — 공개 익스플로잇 "재현"이 아니다.

미공개 일반화

모델 폴백 체인(sonnet-5 → opus-4-8)이 처음 보는 유형을 추론으로 공략. 실측 **포팅 미공개 8/8** 발견.

최소 PoC

퍼저의 **축약된 반례**에서, 호출하는 인터페이스만 선언한 **self-contained** `run(address)` PoC를 합성.

에이전트가 **소스로부터 직접 도출**한 경로이고(암기·카피 불가), 처음 보는 타깃으로 **일반화**되며, 산출물은 **재현 가능한 최소 PoC**다.

6/6

공개셋
(취약 4 발견 + 멀쩡 2 기권)

8/8

포팅 미공개 유형
(클라우드 폴백)

14/14

종료코드 정확
(발견 0 / 기권 1)

100%

결정론
바이트 동일 재현

항목	결과	근거
공개셋 4종 취약점	전부 오프라인 발견	재진입·퍼저·휴리스틱, 네트워크 차단 하에서도 PROVEN
멀쩡한 타깃 2종	오탐 0 · 기권	예산 소진 후 exit 1, 거짓 성공 없음
미공개 유형 8종	전부 예산 내 발견	클라우드 추론 일반화 (파일·정답 접근 없음)
결정론	바이트 동일	오프라인 N=3 재생성 + 클라우드 캐시 리플레이
채점 조건	준수	Foundry v1.0.0 · 하네스 byte-identical · validate-submission PASS

06 왜 AutoPoC인가

■ 증명 기반

"의심"이 아니라 **실행되는 PoC**. 하네스가 깨져야만 성공 — 사람이 다시 검증할 필요가 없다.

■ 캐시 결정론

같은 입력 → **바이트 동일** 결과. LLM 비결정성과 무관하게 재현·감사 가능.

■ 오탐 0

멀쩡한 타깃엔 기권. 리포트의 모든 성공이 **실제 성립**임을 보장.

■ 스스로 도출

파일·정답 접근 없이 소스에서 직접 유도 — 공개 익스플로잇 카피가 아니다.

■ 미공개 일반화

폴백 체인으로 처음 보는 유형까지 공략, 포팅 8/8.

■ 최소 PoC

축약 반례 기반 self-contained `run(address)`.

실행되는 PoC = 신뢰. **AutoPoC는 취약점을 주장하지 않고 증명한다.**